TECHNICAL AUDIT — READ-ONLY REVIEW

StealthTag: What's Actually Built

A file-by-file audit of the current repository against the intended ERC-5564 / ERC-6538 / ERC-4337 privacy-payments architecture. Every claim below is tied to a specific file, and every "broken" claim was verified against the package actually installed in `node_modules`, not against what `package.json` says should be there.

Repo: stealthtag/ (Next.js 16, single Git commit) **Audited:** 2026-08-30

Method: full source read + installed-dependency cross-check

- Landing page — build-breaking bug
- Stealth crypto — crashes at runtime
- AA / Paymaster sweep — crashes at runtime
- Registry / Announcer libs — correct, unused
- Backend — none exists
- Tests — none exist

CONTENTS

Executive summary

StealthTag's *documented* design is a legitimate ERC-5564 + ERC-6538 + ERC-4337 stealth-payment architecture. The *repository* is a Next.js app containing one working page (a marketing landing page — which itself fails to build), a complete-looking but non-functional library layer, and nothing that connects the two.

THE SINGLE MOST IMPORTANT FINDING

There is no page in this app other than the landing page. `app/setup`, `app/send`, `app/scan`, `app/explore` — all described in the README, all linked from the header nav and the homepage CTAs — do not exist as files. Every nav link and CTA button 404s. The entire `lib/` and `hooks/` layer (stealth derivation, registry, announcer, smart account, sweep) is **never imported by any page or component**. It is dead code from the running app's perspective.

What has actually been implemented: a landing page describing the product; a fully-typed data model (`types/index.ts`); library functions for meta-address encoding, ERC-5564 derivation/detection, ERC-6538 registry read/write, Announcer publish/scan, and an ERC-4337 Kernel-v3 sweep via Pimlico; three React hooks wrapping those libraries; a demo-mode fallback with hardcoded fake data.

What is working: the landing page's copy and layout are complete — but it fails to render because it imports a package (`lucide-react`) that is not installed. Nothing else is reachable through the UI, so "working" in the sense of "a user can click through it" is currently zero features.

What is mocked/stubbed: demo mode (`lib/demo.ts`, `useScanner`'s demo branch, `simulateSweep`) fabricates payments and a fake sweep transaction hash. Its own code comment claims stealth detection "remains real" in demo mode — that claim is false; the demo path in `useScanner.ts` explicitly skips detection and returns a single hardcoded dummy private key (`0x00…01`) for every payment.

What is missing: every screen (setup, send, scan, explore), any wiring between hooks and UI, a working import of the ERC-5564 SDK (a core symbol it imports does not exist in the installed package and crashes at runtime), a working import of the AA library (three core symbols it

imports do not exist in the installed `permissionless` version), any backend, any smart contract of StealthTag's own, and any tests.

END-TO-END FLOW?

Not implemented. No page can register a handle, send a payment, scan, or sweep. Even calling the underlying functions directly (e.g. from a script) fails: the ERC-5564 derivation path throws immediately on an undefined import, and the AA sweep path throws immediately on three undefined imports.

DEMO-READY?

No, not as committed. The one page that exists cannot render (missing dependency). Fixing that alone would only show a marketing page — none of the four flow screens exist to click through.

IS THE PRIVACY CLAIM TECHNICALLY DEFENSIBLE?

Not yet, and not because the design is wrong — the ERC-5564/6538 *design as documented* is standard and directionally sound. It is indefensible today because there is no functioning system to make the claim about: the code paths that would *deliver* unlinkability (stealth address derivation, viewing-key detection, sponsored sweep) do not execute successfully in their current form. A privacy claim needs a working mechanism behind it, not just a correct diagram.

Confidence assessment

LAYER	ASSESSMENT	WHY
Product architecture / design docs	• Sound on paper	README correctly describes ERC-5564/6538/4337 roles and an honest "what we don't provide" section.
Frontend (UI you can use)	• Not implemented	Only a landing page exists, and it fails to build.
Stealth cryptography (`lib/stealth.ts`)	• Prototype / broken	Correct algorithm intent, wrong SDK symbol — throws at runtime.
ERC-6538 registry / Announcer libs	• Correct but inert	ABIs match the real standard; never called by anything.

ERC-4337 / Paymaster sweep	• Prototype / broken	Imports functions that don't exist in the installed `permissionless` version.
Backend	• Not implemented	No API routes exist; none are needed by the current design, but none exist to audit either.
Tests	• Not implemented	Zero test files anywhere in the repo.

02 / 21

Repository map

This is the entire repository — 21 source files. There is no `app/setup`, `app/send`, `app/scan`, `app/explore`, no `components/stealth/`, no `ARCHITECTURE.md`/`DEMO.md`/`PITCH.md`, despite all being listed in the README's "Project Structure" tree. Files below are grouped as requested; several requested groups (Smart Accounts as a distinct layer, Paymaster, Relayer/Bundler, Database, Tests, Demo infra beyond one file) have **no dedicated files** — that absence is itself the finding for those groups.

Frontend

`app/page.tsx` • Broken at build

Purpose	Landing page — hero, "how it works" cards, privacy comparison, AA explainer.
Depends on	`lucide-react` (5 icons), `next/link`
Called by	Next.js router (`/`)
Calls	Nothing from `lib/` or `hooks/` — 100% static content
Status	`lucide-react` is imported but is absent from `package.json` , `package-lock.json` , and `node_modules` . This throws a module-resolution error.
Privacy	None — purely marketing copy, but two of its CTAs (`href="/setup"` , `href="/send"`) link to routes that don't exist.

`app/layout.tsx` • Functional

Purpose	Root HTML shell, font, metadata, wraps children in `Providers` + `Header` + footer.

Calls	`app/providers.tsx`, `components/layout/Header.tsx`
Status	Structurally fine; inherits the missing-`lucide-react` failure via `Header`.

`app/providers.tsx` • Unverified compatibility

Purpose	Wraps app in `WagmiProvider` → `QueryClientProvider` → `RainbowKitProvider`.
Depends on	`@rainbow-me/rainbowkit` 2.2.11, `wagmi`, `@tanstack/react-query`
Status	RainbowKit 2.2.11's declared peer dependency is `wagmi ^2.9.0`; the repo installs `wagmi 3.7.7`. See §15.

`components/layout/Header.tsx` • Broken at build; dead links

Purpose	Sticky nav bar with logo, 4 nav links, RainbowKit `ConnectButton`.
Depends on	`lucide-react` (missing), `@rainbow-me/rainbowkit`
Status	`NAV_LINKS` (lines 8–13) point at `/setup`, `/send`, `/scan`, `/explore` — none exist. Every nav item 404s even if the missing dependency were fixed.

`components/ui/index.tsx` • Broken at build; unused

Purpose	Design-system primitives: `Card`, `Badge`, `Button`, `TxStatusBadge`, `AddressPill`, `SectionHeading`, `AlertBox`, `StepNumber`, `Spinner`, `Divider`.
Depends on	`lucide-react` (missing)
Called by	**Nothing.** No page or component imports from `@/components/ui`. This is a ready-made component kit for the four screens that were never built.

Hooks

`hooks/useStealthKeys.ts` • Orphaned + downstream crash

Purpose	Generate/load/clear a recipient's stealth key bundle via two wallet signatures; persists to `localStorage`.
Calls	`lib/stealth.ts#generateStealthKeyBundle` (this itself is fine — see §4), wagmi's `useSignMessage`
Called by	**No page.** The setup screen that would call this does not exist.
Status	Would run today (its own SDK usage doesn't touch the broken symbol), but is unreachable from the UI.

`hooks/useScanner.ts`

• Orphaned; demo path is misleading

Purpose	Fetch announcements, run view-tag + ECDH detection, attach balances.
Calls	`lib/announcer.ts#fetchAnnouncements`, `lib/stealth.ts#scanAnnouncements` (crashes — see §4/§6), `lib/demo.ts`
Called by	**No page.** The scan screen does not exist.
Status	Real-mode branch would throw immediately via `scanAnnouncements`. Demo-mode branch (lines 60–73) does not call detection at all — it fabricates `DetectedPayment` objects directly from `DEMO_PAYMENTS`, using a single hardcoded stealth private key (`0x000…001`) for every entry, contradicting `lib/demo.ts`'s own comment that "detection… remains real."

`hooks/useSweep.ts`

• Orphaned + downstream crash

Purpose	Drives the sponsored-sweep UX state machine (idle → pending → submitted → confirmed/failed).
Calls	`lib/smartAccount.ts#sponsoredSweep` (crashes — see §8/§9), `#simulateSweep`, `lib/demo.ts`
Called by	**No page.** The scan/sweep screen does not exist.

Stealth cryptography / ERC-5564

`lib/stealth.ts`

• Crashes at runtime

Purpose	Meta-address encode/parse, key-bundle generation from wallet signatures, sender-side derivation, recipient-side detection.
Key fns	`generateStealthKeyBundle`, `encodeMetaAddress`, `parseMetaAddress`, `deriveStealthAddress`, `detectPayment`, `scanAnnouncements`, `encodeAnnouncerMetadata`
Depends on	`@scopelift/stealth-address-sdk`, `viem`
Called by	`hooks/useStealthKeys.ts`, `hooks/useScanner.ts`, `lib/announcer.ts`
Status	Imports `StealthAddressType` from the SDK (line 21) — this export does not exist anywhere in the installed package (verified against its `.d.ts` files; the real enum is `VALID_SCHEME_ID` with member `SCHEME_ID_1`, and it is not re-exported under any alias). Every call site that reads `StealthAddressType.SCHEME_ID_1` — `deriveStealthAddress` (line 174) and `detectPayment` (lines 218, 229) — throws `TypeError: Cannot read properties of undefined`. See §4 and §6 for the full breakdown, including a separate, independent parameter-shape bug in the `checkStealthAddress` call.

Privacy	Highest — this is the file that would deliver stealth-address unlinkability. It does not currently work.

`lib/announcer.ts` • Correct, unused

Purpose	Publish announcements to the ERC-5564 Announcer; fetch/parse `Announcement` logs.
Key fns	`publishAnnouncement`, `fetchAnnouncements`, `getLatestBlock`
Called by	`hooks/useScanner.ts` (only path that reaches it, and only page-wired path is missing)
Calls	`lib/chain.ts`'s `ANNOUNCER_ABI`, `lib/stealth.ts#encodeAnnouncerMetadata`
Status	ABI and event shape match the real ERC-5564 Announcer contract. Logically sound, but nothing ever calls `publishAnnouncement` — the sender flow that would call it (the send page) doesn't exist, so payments would never actually get announced even if a stealth address were correctly derived.

ERC-6538

`lib/registry.ts` • Correct, unused

Purpose	Register and resolve stealth meta-addresses against the ERC-6538 Registry.
Key fns	`registerMetaAddress`, `resolveMetaAddress`, `hasMetaAddress`
Called by	**Nothing.** No setup page to register, no send page to resolve.
Status	Function names (`registerKeys`, `stealthMetaAddressOf`) and event (`StealthMetaAddressSet`) match the canonical ERC-6538 Registry interface. This is the most standards-correct file in the repo — and it is never invoked.

ERC-4337 / Smart accounts / Paymaster

`lib/smartAccount.ts` • Crashes at runtime

Purpose	Create a Kernel-v3 smart account, wire a Pimlico bundler + verifying paymaster, execute a sponsored sweep.
Key fns	`createKernelSmartAccount`, `getSmartAccountAddress`, `sponsoredSweep`, `simulateSweep`, `formatEthBalance`
Depends on	`permissionless` 0.4.0 (top-level, `/accounts`, `/clients/pimlico`)
Called by	`hooks/useSweep.ts` (unreachable — no page)

Status	Imports `ENTRYPOINT_ADDRESS_V07` from `permissionless`, `signerToEcdsaKernelSmartAccount` from `permissionless/accounts`, and `createPimlicoBundlerClient` / `createPimlicoPaymasterClient` from `permissionless/clients/pimlico` — **none of these four exist** in the installed package (verified against its actual export list). The real API surface uses `toEcdsaKernelSmartAccount` (different parameter shape) and a single `createPimlicoClient`. Every function in this file that touches these imports fails immediately. See §8/§9.
Privacy	Highest — this is the file the README calls "load-bearing" for privacy. It does not currently work.

Configuration

`lib/chain.ts` • Functional

Purpose	Contract addresses (canonical ScopeLift Sepolia deployments), minimal ABIs, Sepolia public client factory, Pimlico URL builders, explorer URL helpers, demo-mode flag logic.
Called by	Every other `lib/` file
Status	No functional bugs found; ABI shapes match the real contracts (cross-checked in §4/§5).

`lib/wagmi.ts` • Functional but version-mismatched

Purpose	`getDefaultConfig` for wagmi/RainbowKit, Sepolia-only, WalletConnect project ID from env.
Status	Falls back to the literal string `'DEMO_PROJECT_ID'` if no WalletConnect project ID is set — this is not a valid WalletConnect Cloud project ID, so wallet connection via WalletConnect (not injected wallets) would fail silently or error at runtime with an unconfigured env.

`lib/demo.ts` • Cosmetic mock data

Purpose	Hardcoded fake "payments," fake sweep-hash generator, artificial delay simulators.
Status	`DEMO_PAYMENTS` addresses/ephemeral keys are arbitrary hex filler, not real secp256k1 output — they are not derived from `DEMO_META_ADDRESS` by any code path. The file's own header comment ("stealth derivation and detection... REAL even in demo mode") is not accurate for the consumer that actually uses it (`useScanner.ts`'s demo branch bypasses detection entirely).

types/index.ts • Functional

Purpose	Shared TS interfaces: `StealthMetaAddress`, `StealthKeyBundle`, `DerivedStealthAddress`, `AnnouncementEvent`, `DetectedPayment`, `TxState`, `SmartAccountInfo`, `DemoPayment`.
Status	Internally consistent, but `DerivedStealthAddress.viewTag: number` does not match the SDK's actual return type (`HexString`) — see §6.

GROUPS WITH NO FILES AT ALL

Smart Accounts (as a distinct on-chain artifact), **Paymaster**, **Relayer/Bundler**, **Backend/API**, **Database**, **Tests**, and dedicated **Demo/mock infrastructure** beyond `lib/demo.ts` have zero files in this repository. StealthTag reuses ScopeLift's canonical Sepolia contract addresses and Pimlico's hosted bundler/paymaster service by URL — it deploys nothing of its own.

---

03 / 21

Actual end-to-end flow

Tracing what the *current* code does, step by step, not what the README describes.

User registration

1 **Stealth meta-address generation**

• Code exists — hooks/useStealthKeys.ts#generateKeys → lib/stealth.ts#generateStealthKeyBundle. Deterministic: keccak256 of a wallet signature over a fixed string becomes the private key.

2 **ERC-6538 registration**

• Not reachable — lib/registry.ts#registerMetaAddress exists and targets the correct contract, but no screen calls it. • NOT IMPLEMENTED as a user-facing step.

3 **Handle resolution**

• NOT IMPLEMENTED — there is no concept of a human-readable "handle" anywhere in the code (no ENS integration, no username registry). "Handle" in the README means the raw meta-address string itself.

Sender

1. ### Enter handle / resolve meta-address

   • NOT IMPLEMENTED — no send page exists to accept input.

2. ### Derive stealth address (ECDH)

   • Broken — `lib/stealth.ts#deriveStealthAddress` calls the SDK's `generateStealthAddress` with `schemeId: StealthAddressType.SCHEME_ID_1`. `StealthAddressType` is `undefined`. Throws immediately.

3. ### Send payment

   • NOT IMPLEMENTED — no code anywhere constructs or sends the actual value-transfer transaction to the derived stealth address. Only the address derivation and the announcement-publish exist as separate library calls; nothing wires "send ETH, then announce" together.

4. ### Announcement

   • Library exists, unreachable — `lib/announcer.ts#publishAnnouncement` is correct against the ERC-5564 Announcer ABI but is never called by anything reachable from step 2 (which crashes first anyway).

Recipient

1. ### Scanner watches announcements

   • Library exists, unreachable — `lib/announcer.ts#fetchAnnouncements`, real-mode path in `hooks/useScanner.ts`. No scan page to trigger it.

2. ### View-tag filtering

   • NOT IMPLEMENTED as a separate optimization — `lib/stealth.ts` does not pre-filter by view tag before calling `checkStealthAddress`; it delegates the whole check to the SDK in one call per announcement (and doesn't even pass the view tag — see §4). The README's claimed "256× cheaper scanning" step does not exist in the code as written.

3. ### ECDH detection

   • Broken — `detectPayment` calls `checkStealthAddress` with the wrong field name (`stealthAddress` instead of the SDK's required `userStealthAddress`) and omits the required `viewTag` field, on top of the same `StealthAddressType` crash as the sender path.

4. ### Stealth private key derivation

• Broken — same file, same crash; `computeStealthKey` is never reached because `detectPayment` throws before it.

Sweep

1. **Stealth account / UserOperation construction**

   • Broken — `lib/smartAccount.ts#sponsoredSweep` calls `createKernelSmartAccount`, which imports four symbols that don't exist in the installed `permissionless` package. Throws on the first call.

2. **Bundler / Paymaster / EntryPoint**

   • Never reached — code never executes far enough to make a network call to Pimlico.

3. **Destination wallet**

   • NOT IMPLEMENTED — no screen exists to pick or confirm a destination address for swept funds.

WHAT AN EXTERNAL BLOCKCHAIN OBSERVER CAN CURRENTLY SEE

Nothing, because nothing runs. If every bug above were fixed and the missing screens were built, an observer would see: an ERC-6538 `StealthMetaAddressSet` event linking an EOA to a meta-address (public registry, permanently linkable to that EOA — see §5), an ERC-5564 `Announcement` event per payment (ephemeral pubkey + view tag, publicly linkable to the sender's `caller` address and the stealth address), a plain ETH transfer from sender to stealth address (amount and sender visible), and — once the AA path is fixed — a UserOperation execution from the stealth-address-derived smart account, sponsored by Pimlico's paymaster (see §9/§10 for what this reveals).

---

04 / 21

ERC-5564 audit

Checking the implementation against the standard, requirement by requirement.

REQUIREMENT	IMPLEMENTATION	FILE / FN

Meta-address structure	Concatenated 65-byte uncompressed spending + viewing pubkeys, `st:eth:0x…` prefix	`lib/stealth.ts#encodeMetaAddr`
Spending public key	Derived from `keccak256(spendingSignature)` via `privateKeyToAccount`	`lib/stealth.ts#generateStealt`
Viewing public key	Same pattern, separate fixed message	same
Ephemeral key generation	Delegated to SDK's `generateStealthAddress` (generates internally if not supplied)	SDK, called from `deriveStealthAdd`
ECDH	Delegated to SDK	SDK internals
Shared secret / KDF	Delegated to SDK (`getHashedSharedSecret`)	SDK internals
Stealth pubkey derivation	Delegated to SDK	SDK internals
Stealth address derivation	`deriveStealthAddress()` calls SDK with `schemeId: StealthAddressType.SCHEME_ID_1`	`lib/stealth.ts:169-181`
Announcement	`announce(schemeId, stealthAddress, ephemeralPubKey, metadata)`	`lib/announcer.ts#publishAnnou`
Announcer contract	Canonical ScopeLift Sepolia deployment, not custom	`lib/chain.ts CONTRACT_ADDRESS`
Metadata format	`0x01<viewTagByte>`	`lib/stealth.ts#encodeAnnounce`
View tags	Extracted from metadata byte 1 on scan; not used for pre-filtering before the full ECDH check	`lib/stealth.ts#detectPayment`
Recipient scanning	Iterates all fetched announcements, calls detection on each	`lib/stealth.ts#scanAnnounceme`
Stealth private key derivation	`computeStealthKey({ephemeralPublicKey, spendingPrivateKey, viewingPrivateKey, schemeId})`	`lib/stealth.ts#detectPayment:`

IS THE SCOPELIFT SDK USED CORRECTLY?

No. Two of the five imported symbols from `@scopelift/stealth-address-sdk` do not exist in the installed `1.0.0-beta.5` package: `StealthAddressType` (real name: `VALID_SCHEME_ID`) and `parseStealthMetaAddress` (real name: `parseStealthMetaAddressURI`, and it's imported-but-unused so it's silently wrong rather than crashing). The `checkStealthAddress` call additionally passes a field named `stealthAddress` where the SDK requires `userStealthAddress`, and omits the SDK's required `viewTag` field entirely. This is not a subtle edge case — it is every call site in the file that touches scheme selection or address verification.

---

05 / 21

ERC-6538 audit

The identity is the connected EOA address (`walletClient.account`), mapped 1:1 to one meta-address per scheme ID. **Registration** is a single `registerKeys(schemeId, stealthMetaAddressBytes)` write, gas-paid by the user's own EOA (no sponsorship for this step). **Storage** is entirely on-chain, in the canonical ERC-6538 Registry contract at `0x6538E6bf4B0eBd30A8Ea093027Ac2422ce5d6538` on Sepolia — not a custom contract, not a database. **Resolution** is a public `stealthMetaAddressOf(registrant, schemeId)` view call, callable by anyone with an RPC connection, no auth.

QUESTION	ANSWER
Real registry or custom approximation?	• Real — reuses ScopeLift's canonical deployment; ABI matches (`registerKeys`, `stealthMetaAddressOf`, `StealthMetaAddressSet` event)
Is the "handle" on-chain?	Yes — the entire meta-address (both public keys) is stored as raw bytes in registry storage, publicly readable forever
Can anyone query it?	Yes, trivially — it is a public `view` function; no authentication or gating exists or is intended
Reachable from the UI today?	• No — `registerMetaAddress` / `resolveMetaAddress` are never called by any component

PRIVACY INFORMATION THE REGISTRY REVEALS

By design (this is inherent to ERC-6538, not a StealthTag bug): registering permanently and publicly links a real-world EOA to that person's stealth spending/viewing public keys. Anyone can enumerate every address that has ever registered, and correlate registration timing with the first stealth payment received. This is expected and matches the standard — StealthTag's README acknowledges "sender anonymity" and full metadata privacy are out of scope, but it does not explicitly call out that *registration itself* is the one irreversible, permanent link between a public identity and the stealth system. Worth stating plainly to judges.

---

06 / 21

Cryptography audit

All elliptic-curve math is delegated to the ScopeLift SDK (secp256k1, standard generator/order — nothing hand-rolled, which is the right call). The bugs are entirely in how StealthTag's own code *calls* that SDK, plus one independent key-derivation weakness.

BREAKING

Undefined enum reference crashes every stealth-address operation

`lib/stealth.ts` imports `StealthAddressType` from `@scopelift/stealth-address-sdk`. It does not exist in the package (confirmed against the installed `.d.ts` and compiled JS — the actual exported enum is `VALID_SCHEME_ID`). `StealthAddressType.SCHEME_ID_1` therefore evaluates as `undefined.SCHEME_ID_1`, throwing at every call to `deriveStealthAddress` and `detectPayment`.

BREAKING

Wrong parameter shape passed to `checkStealthAddress`

The SDK requires `{ userStealthAddress, viewTag, ephemeralPublicKey, spendingPublicKey, viewingPrivateKey, schemeId }`. The code passes `stealthAddress` (not a field the function reads) and never supplies `viewTag` at all. Independent of the crash above, this call would silently misbehave — the function would be checking against `undefined` fields — even if the enum bug were fixed.

CORRECTNESS

View tag type mismatch: hex string treated as a number

The SDK's `GenerateStealthAddressReturnType.viewTag` is typed `HexString` (e.g. `"0xab"`). StealthTag's `DerivedStealthAddress.viewTag` is typed `number`, and `encodeAnnouncerMetadata(viewTag: number)` calls `viewTag.toString(16)` on it. `String.prototype.toString` ignores a radix argument and just returns the string unchanged — so the "metadata" byte written on-chain would be malformed (e.g. literally `0x010xab`-shaped garbage), not a clean single byte. This would silently corrupt every announcement's metadata field even after the crash above is fixed.

DESIGN WEAKNESS

Deterministic keys from a fixed, public signature message

Spending and viewing private keys are `keccak256(signature)`, where the signed message is a hardcoded literal string identical for every user of the app (`"StealthTag: Generate Stealth Spending Key v1"` / `"...Viewing Key v1"`, `lib/stealth.ts:41-42`). This is convenient (keys are always regenerable from the wallet, no backup needed) but means anyone who can get a user's EOA to sign these two exact strings — e.g. a malicious dApp requesting an "innocuous" personal-sign — can fully reconstruct both stealth keys without ever touching the wallet's actual private key. See §18.

DOCUMENTATION

Misleading code comment

`hooks/useStealthKeys.ts:12-13` claims keys are "stored in localStorage (encrypted by the wallet's address as a simple namespace)." The address is used purely as a storage-key *prefix*, not as encryption input — the stored JSON blob (including both private keys) is plaintext. See §7.

WHAT'S ACTUALLY RIGHT

Curve choice (secp256k1), key format (uncompressed 65-byte public keys, standard Ethereum address derivation via `viem/accounts`), and the high-level derivation formulas documented in the README (`S = e·K_s`, `h = keccak256(S)`, `P = h·G + K_s`, `p = h + s`) are all standard and match ERC-5564. The failure is entirely in the glue code calling the SDK, not in the cryptographic design itself.

Key management

QUESTION	ANSWER
Spending private key location	Browser `localStorage`, plaintext, key `stealthtag_keys_<address>` — `hooks/useStealthKeys.ts`
Viewing private key location	Same object, same storage, plaintext
Public keys location	Same object (redundant with the private keys, from which they're derivable), plus published on-chain via ERC-6538 registration
Does the backend ever see private keys?	• **No backend exists** — nothing to leak to server-side
Does the frontend hold private keys?	Yes — in memory (React state) and in `localStorage`
localStorage?	• **Yes, plaintext** — both private keys
sessionStorage / cookies?	Not used
Database?	No database exists
Environment variables?	No key material in env — only RPC/Pimlico/WalletConnect API keys (see §16)
Passed to any API?	No API exists to pass them to
Can a viewing-only watcher detect payments without spending ability?	By design, yes (`checkStealthAddress`/`computeStealthKey` take the viewing key separately) — but this can't currently be exercised because detection crashes before the viewing/spending split matters in practice
Can a watcher spend funds?	No — `detectPayment` only *returns* the stealth private key to the caller holding the full `StealthKeyBundle` (which includes the spending key); the SDK's `checkStealthAddress` function itself only needs the viewing key + spending *public* key to detect, and does not require the spending private key

IS VIEWING = DETECTION / SPENDING = SPENDING ACTUALLY IMPLEMENTED?

Partially, at the type level: `StealthKeyBundle` and the SDK's function signatures correctly separate the two keys, and `detectPayment` passes only `viewingPrivateKey` (plus the public spending key) into `checkStealthAddress`. But because the whole bundle — spending private key included — is generated and stored together in one object on one device with no code path that ever hands out a viewing-only bundle to a separate "watcher" identity, the separation exists in the data model but is never *exercised* as a distinct capability anywhere in the app. There is no "view-only mode," no way to export just the viewing key, no separate watcher UI.

---

08 / 21

ERC-4337 / Account Abstraction audit

Intended architecture: Kernel v3 (ZeroDev) smart account via `permissionless`, EntryPoint v0.7, Pimlico bundler + verifying paymaster, Sepolia.

QUESTION	ANSWER
Smart account implementation	Kernel v3, intended via `signerToEcdsaKernelSmartAccount`
Which EntryPoint	v0.7, intended via the constant `ENTRYPOINT_ADDRESS_V07`
Bundler	Pimlico, intended via `createPimlicoBundlerClient`
Network	Sepolia (11155111)
Is the bundler real or mocked?	Would be a real hosted service (Pimlico) *if reached* — but the code never gets there
Is the smart account actually deployed?	• No — counterfactual deployment on first UserOp is the intended pattern, but no UserOp is ever successfully constructed
Can it execute a sweep?	• No

WHY IT DOESN'T WORK: DEPENDENCY-VERSION DRIFT, NOT LOGIC ERRORS

`lib/smartAccount.ts` is written against an older-generation `permissionless` API. The installed package (`permissionless@0.4.0`, per `package.json`'s own `^0.4.0` range) actually exposes a newer surface:

IMPORTED (CODE)	EXISTS IN INSTALLED PACKAGE?	ACTUAL EXPORT
`ENTRYPOINT_ADDRESS_V07` from `permissionless`	• No	Not exported at all — `permissionless` top-level exports include `createSmartAccountClient` (this one *is* correct) but no entry-point constants
`signerToEcdsaKernelSmartAccount` from `permissionless/accounts`	• No	`toEcdsaKernelSmartAccount` (different name, different parameter shape — takes `owners: [...]`, not a single `signer`)
`createPimlicoBundlerClient` from `permissionless/clients/pimlico`	• No	Only `createPimlicoClient` exists (combined bundler+paymaster client)
`createPimlicoPaymasterClient` from `permissionless/clients/pimlico`	• No	Same — folded into `createPimlicoClient`

A named import of a non-existent export resolves to `undefined` in this module setup; every subsequent call (`ENTRYPOINT_ADDRESS_V07` used as a value, `signerToEcdsaKernelSmartAccount(...)` called as a function) throws immediately. This affects *every* function in the file, real mode and would-be counterfactual address lookups alike.

Exact transaction flow, as coded (if the imports were fixed): stealth private key → `privateKeyToAccount` → used as the *signer* for a brand-new Kernel smart account counterfactually deployed *at an address derived from the stealth key itself* → `smartAccountClient.sendTransaction({to, value: fullBalance, data:'0x'})` → paymaster-sponsored UserOp → EntryPoint → transfer executes, deploying the Kernel account in the same UserOp. This design detail is worth double-checking once fixed: it deploys a *new* counterfactual smart account per stealth address (since each stealth address is a fresh signer), not one persistent account — meaning the "smart account" isn't the recipient's one identity, it's a

throwaway wrapper created fresh at each stealth address purely to route the sponsored transfer out.

---

09 / 21

Paymaster audit

QUESTION	ANSWER
Does a Paymaster contract exist?	Not one StealthTag deploys — it targets Pimlico's hosted verifying paymaster service by URL
Who funds it?	Pimlico, funded by whoever holds the Pimlico account behind the API key in `.env.local` (the developer, not disclosed/deployed in this repo)
Does the user send money to the Paymaster?	No
Does the stealth wallet send money to the Paymaster?	No
Is gas sponsorship real or simulated?	• **Neither — currently non-functional.** Would be real (via Pimlico) if the `permissionless` import bugs (§8) were fixed and a funded API key supplied. Falls back to a fully simulated fake tx hash (`generateDemoSweepHash`) whenever `NEXT_PUBLIC_PIMLICO_API_KEY` is absent — which is also the current, uncorrected default.
How is the paymaster selected?	Hardcoded to Pimlico's URL pattern in `lib/chain.ts#getPimlicoUrl`; no selection logic, no fallback paymaster
How does it validate UserOps?	Delegated entirely to Pimlico's hosted service — no custom validation logic in this repo
Deposit/stake with EntryPoint?	Not this repo's concern — that's Pimlico's infrastructure, opaque to StealthTag's code

What happens when it runs out of funds?	Not handled — no retry, no fallback, no user-facing error path distinguishing "paymaster rejected" from any other failure; `sponsoredSweep` would just throw and `useSweep` would show a generic "Sweep failed" message

VISIBLE RELATIONSHIP: USER WALLET → PAYMASTER → RELAYER → STEALTH ADDRESS

Yes, structurally — and this is expected/acceptable for this architecture, not a bug: the same Pimlico API key is used for every user's bundler and paymaster traffic. That does not, by itself, link one user's stealth addresses to their public wallet on-chain (the chain only sees "EntryPoint executed a UserOp, paymaster X sponsored it" — the paymaster's identity is public but generic, shared across all of Pimlico's customers, not just StealthTag). The real correlation risk is at the network/metadata layer, covered in §10/§11 — not from the paymaster's on-chain footprint itself.

10 / 21

Relayer / bundler privacy audit

"Paymaster" is not automatically "privacy." Checking the actual wiring.

QUESTION	ANSWER
Is there a relayer?	No custom relayer — the frontend would call Pimlico's public bundler RPC directly
Is there a bundler?	Yes, Pimlico's hosted bundler (intended; currently unreachable due to §8's import bugs)
Normal public RPC or custom relay API?	Sepolia reads go through whatever RPC URL is in `NEXT_PUBLIC_RPC_URL` (Alchemy/Infura per `.env.example`) — a standard third-party RPC, not a privacy-preserving relay
Does the frontend contact the bundler directly?	Yes — `lib/smartAccount.ts` constructs the bundler/paymaster clients client-side, in the browser, with the Pimlico API key exposed via `NEXT_PUBLIC_*` (client-visible by design in Next.js)
Does a backend relay anything?	No backend exists (see §13)

Does the relayer/paymaster know the stealth address?	Yes, necessarily — it's the `sender` of the UserOperation it's bundling/sponsoring
Does it know the user's public wallet?	Not directly from the sweep UserOp itself (the signer is the stealth key, not the EOA) — but see the browser-session correlation note below

THE IP/SESSION CORRELATION GAP

Because there is no relay layer, every RPC call the user's browser makes — resolving a handle from the Registry, fetching Announcer logs to scan, and (once fixed) submitting sweep UserOps to Pimlico — originates from the *same browser session and IP address*. An RPC provider or bundler operator sitting in the middle can trivially correlate "this IP registered meta-address M," "this IP scanned the Announcer around the same time stealth address S received funds," and "this IP submitted a sweep UserOp moving funds out of S" — even though none of that correlation is visible on-chain. The README's own "Future Improvements" section names this ("Private RPC relay — prevent IP-level linkage") as unbuilt, which is accurate. This is the single largest privacy gap in the current design, independent of all the runtime bugs.

Classification

Current architecture (as designed, ignoring the runtime bugs): Class A — Basic stealth-address privacy. On-chain unlinkability of receipt addresses is the design goal and, if the bugs were fixed, would be delivered by ERC-5564 alone. There is no relayer-layer privacy (Class C) and no additional off-chain privacy tooling like mixnets, private RPC, or Tor routing (Class D) — every network call is made directly, from the user's own browser/IP, to standard third-party infrastructure (Alchemy/Infura/Pimlico) that can see the caller's IP and timing for every request.

Privacy threat model

Assuming a fully fixed, fully deployed version of this exact design (not today's broken build) — what a chain-plus-metadata observer can infer.

HIGH SEVERITY

Registration permanently links EOA → stealth keys

How: ERC-6538 `StealthMetaAddressSet` event, indexed by registrant address. **Who can observe:** anyone, forever, on any block explorer. **Mitigation:** none available within ERC-6538 itself — this is inherent to the standard; the only mitigation is registering from an EOA that is itself already privacy-hardened (not this repo's concern).

HIGH SEVERITY

IP-level correlation via direct RPC/bundler calls

How: every scan, resolve, and (future) sweep submission comes from the user's own browser IP, straight to Alchemy/Infura/Pimlico. **Who can observe:** the RPC/bundler operators; anyone with access to their logs. **Mitigation:** route through a privacy-preserving relay or self-hosted node — explicitly listed as unbuilt in the README's own roadmap.

MEDIUM SEVERITY

Amount correlation across payments

How: transfer amounts are always plaintext on Ethereum; if a sender always pays the recipient the same round amount (e.g. a recurring 0.1 ETH subscription), the amount pattern alone can suggest a link even without address reuse. **Who can observe:** anyone watching the Announcer + corresponding transfers. **Mitigation:** amount splitting (README lists this as a future improvement, unbuilt).

MEDIUM SEVERITY

Consolidation-pattern leakage at sweep time

How: if/when swept funds are all sent to the same destination address (the recipient's real wallet), that destination becomes the de facto link point — an observer who suspects a given EOA is StealthTag's recipient can watch for multiple "fresh smart account → same destination" sweep transactions and reconstruct the aggregate balance retroactively, even though the stealth addresses themselves were unlinkable at receipt time. **Who can observe:** anyone. **Mitigation:** vary destination addresses, delay/randomize sweep timing, batch sweeps unpredictably. None of this exists in `lib/smartAccount.ts` today — `sponsoredSweep` takes an explicit, static `toAddress`.

MEDIUM SEVERITY

Timing correlation between announce and detect/sweep

How: if a recipient's scanner runs shortly after every new Announcement (e.g. a live watcher), timing alone can narrow down which announcements are "interesting" to which client sessions, especially combined with the IP correlation above. **Who can observe:** bundler/RPC operators. **Mitigation:** randomized polling delay, batched scanning, private relay.

LOW SEVERITY

Shared Paymaster fingerprint

How: every StealthTag sweep (once working) would be sponsored by the same Pimlico paymaster contract, distinguishing "a sweep sponsored by this specific paymaster" from ordinary traffic — a weak fingerprint that a sophisticated chain-analytics firm could use to flag "probably a StealthTag sweep," though it does not reveal which stealth addresses belong to which recipient. **Mitigation:** shared/rotating paymasters across many unrelated apps reduce this; not something to fix in-app.

Frontend audit

SCREEN	STATUS	NOTES
Landing page	• Exists, fails to build	Complete copy/layout; crashes on missing `lucide-react`; two CTAs link to nonexistent routes
Identity creation / handle registration (`/setup`)	• Does not exist	Backing hook (`useStealthKeys`) and lib (`registry.ts`) are ready; no UI
Send/payment page (`/send`)	• Does not exist	No UI to enter a handle or trigger a transfer at all
Scan / incoming payments (`/scan`)	• Does not exist	Backing hooks (`useScanner`, `useSweep`) are ready; no UI
Sweep / Paymaster gas sponsorship UI	• Does not exist	Would live on the scan page; no design either, beyond the generic `TxStatusBadge` component
Chain visualization / unlinkability explorer (`/explore`)	• Does not exist	Linked in nav; zero backing code of any kind, not even a stub
Privacy explanation	• Present on landing page only	Two static sections comparing "public address" vs "StealthTag handle" — copy only, no interactivity

Component kit (`components/ui/index.tsx`) is complete and reasonably well-built (badges, tx-status indicator with an explicit "[DEMO]" label for simulated transactions, address pills linking to Etherscan) — but it is entirely unused. Building the four missing screens would mostly be a matter of wiring existing hooks to this existing kit, once the underlying library bugs are fixed.

---

13 / 21

Backend / API audit

NO BACKEND EXISTS.

There is no `app/api/` directory, no server actions, no external service code, nothing under `pages/api` either (this is the App Router, so that pattern wouldn't apply regardless). Every network interaction in the codebase — Sepolia RPC reads/writes, Pimlico bundler/paymaster calls — is made directly from the browser via `viem`/`wagmi`/`permissionless` client instances constructed in `lib/`. This is architecturally consistent with a "no trusted intermediary" privacy story (nothing server-side to compromise or subpoena), but it also means there is no relay layer at all — see §10's IP-correlation finding, which is a direct consequence of this absence.

---

14 / 21

Smart contract audit

StealthTag deploys no contracts of its own. It references two canonical, already-deployed contracts by address.

CONTRACT	ADDRESS	STATUS

ERC-5564 Announcer	`0x55649E01B5Df198D18D95b5cc5051630cfD45564`	• Real deployed contract (ScopeLift) reused, not written by this project
ERC-6538 Registry	`0x6538E6bf4B0eBd30A8Ea093027Ac2422ce5d6538`	• Real deployed contract (ScopeLift) reused, not written by this project
StealthTag Paymaster	—	• Does not exist — uses Pimlico's hosted paymaster, not a StealthTag-deployed contract
StealthTag Smart Account factory	—	• Does not exist as a StealthTag artifa — Kernel v3 is a third-party (ZeroDev) implementation, referenced via `permissionless`, never actually reached (see §8)

Since **no contract in this repo is custom-written or deployed by StealthTag**, there is no bespoke Solidity to audit for reentrancy, access control, or upgrade risk. All contract-level trust is inherited from ScopeLift's audited reference deployments and Pimlico's infrastructure — which is a reasonable choice for a hackathon build, and correctly disclosed in the README ("No custom contracts are deployed").

Dependency audit

PACKAGE	DECLARED	INSTALLED	USED FOR	FLAG
`@scopelift/stealth-address-sdk`	^1.0.0-beta.5	1.0.0-beta.5	ERC-5564 EC math	• Called with 2 none

permissionless	^0.4.0	0.4.0	ERC-4337 smart account, bundler, paymaster clients	• Called with 4 none
viem	^2.56.0	2.56.0	Low-level chain client, ABI encoding, account utilities	• Used correctly thro
wagmi	^3.7.7	3.7.7	Wallet connection hooks	• RainbowKit 2.2.11
@rainbow-me/rainbowkit	^2.2.11	2.2.11	Wallet-connect UI	See wagmi row — its c
@tanstack/react-query	^5.102.8	installed	wagmi's query cache	• Standard, correctly
next	16.3.3	16.3.3	App framework	README claims "Next
lucide-react	— not in package.json —	— not in node_modules —	Icons in page.tsx, Header.tsx, components/ui	• Missing entirely —

NO SECRETS FOUND IN THE REPO

`.env.example` contains only placeholder values (`YOUR_ALCHEMY_API_KEY`, `YOUR_PIMLICO_API_KEY`, etc.) and is correctly excluded from `.gitignore`'s `.env*` pattern alongside it (no `.env.local` is present in the working tree). Nothing to redact.

Network / deployment

ITEM	VALUE	STATUS

Chain	Ethereum Sepolia, chainId `11155111`	Hardcoded in `lib/chain.ts`, l
RPC	`NEXT_PUBLIC_RPC_URL`, fallback `https://rpc.sepolia.org`	• Placeholder in `.env.exampl`
EntryPoint address	Not hardcoded anywhere in this repo	Would come from the (currently `ENTRYPOINT_ADDRESS_V07` co
Announcer address	`0x55649E01B5Df198D18D95b5cc5051630cfD45564`	• Real, hardcoded, matches REA
Registry address	`0x6538E6bf4B0eBd30A8Ea093027Ac2422ce5d6538`	• Real, hardcoded, matches REA
Smart account factory/address	Not hardcoded — computed counterfactually per stealth key via the (broken) Kernel account creation path	• Never resolves today
Paymaster address	Not a contract address — a Pimlico API URL: `NEXT_PUBLIC_PIMLICO_API_KEY` placeholder in `.env.example`	• Placeholder only
Bundler endpoint	Same Pimlico URL pattern, same key	• Placeholder only
WalletConnect project ID	`NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID`, fallback literal `'DEMO_PROJECT_ID'`	• Invalid fallback — not a real W
Deployment scripts	None	• Not applicable — no custom cc

Everything that is "deployed" in this system belongs to a third party (ScopeLift's Announcer/Registry, Pimlico's bundler/paymaster infrastructure). Nothing StealthTag-specific has ever been deployed — there is no deployment history to check, and the single git commit confirms this is pre-deployment.

Testing

ZERO TEST FILES EXIST ANYWHERE IN THE REPOSITORY.

No test runner is configured in `package.json` (no Vitest/Jest script), no `*.test.ts`/`*.spec.ts` files exist. Untested: ECDH, stealth address derivation, view tags, announcements, ERC-6538 register/resolve, scanning, spending-key derivation, UserOperations, smart accounts, Paymaster, relayer, sweeping, privacy properties, and failure cases — all of it, without exception. The two crash-level bugs found in this audit (§4, §8) would have been caught immediately by a single smoke test that called `deriveStealthAddress` or `createKernelSmartAccount` once.

Security audit

CRITICAL

Core crypto and sweep paths are non-functional

`StealthAddressType` and four `permissionless` symbols are undefined imports (§4, §6, §8). Any code path that reaches them throws. This is a functionality-blocking bug, not narrowly a "security" bug, but it means the privacy guarantees the product exists to provide cannot currently be relied on at all.

CRITICAL

Landing page fails to build

Missing `lucide-react` dependency (§2, §12, §15) breaks the only page that exists.

HIGH

Deterministic key derivation from a fixed, public signing message

Anyone able to induce a wallet to sign two known, hardcoded strings can reconstruct both stealth keys (§6, §7). A malicious or compromised dApp requesting an innocuous-looking `personal_sign` could silently harvest a user's entire stealth identity.

HIGH

Plaintext private keys in localStorage

Both stealth private keys sit unencrypted in `localStorage` (§7), readable by any XSS payload, malicious browser extension, or anyone with local disk access to the browser profile. No wrapping key, no passphrase, no OS keychain integration.

MEDIUM

Untested major-version dependency mismatch

RainbowKit 2.2.11 declares a peer dependency on `wagmi ^2.9.0`; the project installs `wagmi 3.7.7` (§15). This combination is outside what either package's authors have tested; wallet connection behavior is unverified.

MEDIUM

No IP-level privacy layer

All RPC/bundler traffic goes directly from the browser to third-party infrastructure (§10) — an operational privacy gap, explicitly acknowledged as unbuilt in the README's own roadmap.

MEDIUM

Full-balance sweep with no reserve/failure handling

`sponsoredSweep` sweeps the entire detected balance (`lib/smartAccount.ts:189`) with no handling for the paymaster rejecting sponsorship mid-flight, no retry, and no partial-sweep fallback — a failed sponsorship after a real transfer isn't distinguished from any other error in the UI (§9).

LOW

Invalid WalletConnect fallback project ID

`lib/wagmi.ts` falls back to the literal string `'DEMO_PROJECT_ID'` if unset (§16) — not a functioning WalletConnect Cloud project, so WalletConnect-based (non-injected) wallet connections would fail without a real ID configured.

Not applicable given current state (would need re-review once the app is functional): replay attacks on UserOperations, signature-malleability handling, reentrancy (no custom contracts exist), nonce management under concurrent sweeps, malicious-announcement spam handling in the scanner (the scanner has no rate limiting or spam filtering built in, worth flagging for the

future), denial-of-service resistance of the (nonexistent) scan UI, and frontend injection surfaces on the (nonexistent) send/scan forms.

Requirement compliance table

REQUIREMENT	EXISTS?	WORKS?	CORRECT?	FILE / FN
Stealth meta-address	Yes	Yes	Yes	`lib/stealth.ts#enco`
Spending key	Yes	Yes	• Weak source	`lib/stealth.ts#gene`
Viewing key	Yes	Yes	• Weak source	same
ERC-5564	Yes	• No	• No	`lib/stealth.ts`
ERC-6538	Yes	• Unreachable	Yes	`lib/registry.ts`
ECDH	Delegated to SDK	• Never reached	N/A	SDK internals
View tags	Yes	• Decorative	• No pre-filter	`lib/stealth.ts#dete`
Unique stealth addresses	Yes (by design)	• No	N/A	`deriveStealthAddres`
Announcer	Yes	• Unreachable	Yes	`lib/announcer.ts`
Scanner	Yes	• No	N/A	`hooks/useScanner.ts`
Payment flow (send)	• No	No	N/A	—

ERC-4337	Yes	• No	• No	lib/smartAccount.ts
Smart account	Yes (code)	• No	N/A	same
EntryPoint	Referenced	• No	N/A	same
Bundler	Referenced (Pimlico)	• No	N/A	same
Paymaster	Referenced (Pimlico)	• No	N/A	same
Relayer	• No	N/A	N/A	—
Privacy-preserving funding	Design intent only	• No	N/A	—
Privacy-preserving sweep	Design intent only	• No	N/A	lib/smartAccount.ts
Frontend	• 1 of 5 screens	• No	N/A	app/page.tsx
End-to-end demo	• No	• No	N/A	—

What should we build next?

Ordered by what actually blocks a demo. Nothing below has been implemented — this is a punch list only.

P0 — ABSOLUTELY REQUIRED

Add the missing `lucide-react` dependency

• P0

Why	The only existing page cannot render without it.
Files	`package.json`, `app/page.tsx`, `components/layout/Header.tsx`, `components/ui/index.tsx`
Expected result	`npm run dev` renders the landing page without errors.
Difficulty	Trivial

Fix the ERC-5564 SDK integration

• P0

Why	This is the core privacy mechanism; it currently crashes on every call.
Files	`lib/stealth.ts`
Expected result	Replace `StealthAddressType` with the real `VALID_SCHEME_ID` enum; fix the `checkStealthAddress` call to use `userStealthAddress` + supply `viewTag`; fix the `viewTag` number/hex-string mismatch feeding `encodeAnnouncerMetadata`. `deriveStealthAddress` and `detectPayment` should run without throwing against real or test data.
Difficulty	Low — the fix is mechanical once the correct SDK API is used; the surrounding logic is already sound.

Fix the ERC-4337 / `permissionless` integration

• P0

Why	The Paymaster-sponsored sweep is what the README calls "load-bearing" for privacy; it currently crashes on every call.
Files	`lib/smartAccount.ts`
Expected result	Migrate to the installed API: `toEcdsaKernelSmartAccount` (owners array, not `signer`), a single `createPimlicoClient` in place of the two separate bundler/paymaster client constructors, and whatever EntryPoint constant/address the current `permissionless/viem` account-abstraction utilities actually export in place of `ENTRYPOINT_ADDRESS_V07`.
Difficulty	Medium — requires reading the current `permissionless` API surface carefully; the counterfactual-account-per-stealth-key design (§8) is also worth reconsidering while touching this file.

Build the four missing screens and wire existing hooks/components

• P0

Why	There is currently no way for a human to use any part of this product.

Files	New: `app/setup/page.tsx`, `app/send/page.tsx`, `app/scan/page.tsx`, `app/explore/page.tsx`. Reuse: `hooks/*`, `components/ui/index.tsx`, `lib/*`.
Expected result	A clickable path: connect wallet → generate keys → register → (separately) send a payment to a handle → scan → sweep.
Difficulty	Medium — the hard logic already exists; this is primarily UI wiring, once P0 items above unblock it.

Implement the missing "send" step

• P0

Why	No code currently connects "derive a stealth address" to "actually transfer ETH there" to "announce it" — three separate library functions that nothing chains together.
Files	New logic in `app/send/page.tsx` or a new `lib/send.ts`, using `lib/stealth.ts#deriveStealthAddress`, a plain `walletClient.sendTransaction`, then `lib/announcer.ts#publishAnnouncement`
Expected result	One user action produces a real Sepolia transfer plus a real Announcer event.
Difficulty	Low-medium

P1 — IMPORTANT

Add a smoke-test suite covering the crypto and AA paths

• P1

Why	Both P0 crashes above would have been caught by a single test each; zero tests currently exist.
Files	New: test files for `lib/stealth.ts`, `lib/smartAccount.ts`; add a test runner to `package.json`
Expected result	CI-runnable proof that derive→detect and sweep construction don't throw.
Difficulty	Medium

Resolve or confirm the wagmi/RainbowKit version mismatch

• P1

Why	Wallet connection is the very first step of every flow; an untested peer-dependency mismatch is a high-risk unknown to leave unresolved before a live demo.
Files	`package.json`
Expected result	Either pin `wagmi` to RainbowKit's supported `^2.9.0` range, or confirm (with a real connect-and-sign test) that 3.7.7 works with 2.2.11.
Difficulty	Low to verify, potentially medium if a downgrade cascades into other API changes

Fix demo mode to make honest claims

• P1

Why	The current demo path fabricates detection results and a single shared dummy private key while its own code comment claims detection is real — misleading if a judge reads the source.
Files	`hooks/useScanner.ts`, `lib/demo.ts`
Expected result	Either make demo mode run real detection against seeded, genuinely-derived demo data, or correct the comments/UI copy to say plainly that both detection and keys are fabricated in demo mode.
Difficulty	Low

Add a real WalletConnect project ID and remove the fallback literal

• P1

Why	`'DEMO_PROJECT_ID'` is not a functioning project — fine for injected wallets (MetaMask), silently broken for anything routed through WalletConnect.
Files	`lib/wagmi.ts`, `.env.local`
Expected result	QR-code/WalletConnect flows work at demo time.
Difficulty	Trivial

P2 — NICE TO HAVE

Implement real view-tag pre-filtering

• P2

Why	The README's headline "256× cheaper scanning" claim currently has no code behind it.
Files	`lib/stealth.ts#detectPayment/scanAnnouncements`
Expected result	A cheap byte comparison rejects most announcements before the SDK's full check runs.
Difficulty	Low, once the SDK call itself is fixed

Vary sweep destinations / add sweep timing jitter

• P2

Why	Reduces the consolidation-pattern leak identified in §11.
Files	`lib/smartAccount.ts`, `hooks/useSweep.ts`
Expected result	Harder for an observer to reconstruct total balance from sweep destinations alone.
Difficulty	Medium — mostly a UX/policy decision, not a technical blocker

Build the unlinkability explorer page

• P2

Why	This is the strongest "wow" moment for judges (visually proving addresses look random) but has zero code today.
Files	New: `app/explore/page.tsx`
Expected result	A visual side-by-side of a public-wallet payment history vs. a StealthTag recipient's scattered stealth addresses.
Difficulty	Low-medium, purely additive once real data exists

---

21 / 21

Final judge-ready verdict

1. Can I demonstrate "one public handle, three payments, three different stealth addresses"?

No, not with the current repository. No send page exists, and the derivation call that would produce those addresses crashes on invocation.

2. Can I demonstrate that an external observer cannot trivially link them?

No — there's nothing on-chain to show an observer failing to link, because nothing gets sent. The design would support this claim once P0 fixes land and real transactions exist to point at.

3. Can I actually sweep funds from a stealth address?

No. `sponsoredSweep` throws immediately on the current `permissionless` install.

4. Is gas sponsorship actually working?

No. It's neither genuinely working nor honestly simulated at the code level right now — it fails before reaching Pimlico, and separately falls back to a fully fake tx hash whenever demo mode is active (which is the effective default with no API key configured).

5. Is the Paymaster genuinely protecting privacy, or only solving gas funding?

By design, both — sponsorship exists specifically so the stealth address is never funded from a known wallet, which is a real privacy mechanism, not just convenience. But today it does neither, because it doesn't run.

6. Is the relayer architecture privacy-preserving?

There is no relayer architecture — the frontend talks directly to third-party RPC/bundler infrastructure from the user's own browser and IP. This is the single biggest privacy gap in the design itself, separate from the implementation bugs, and the README's own roadmap admits it's unbuilt.

7. What is the biggest technical weakness right now?

Two undefined-import crashes (an SDK enum that doesn't exist, and four `permissionless` exports that don't exist in the installed version) sit directly in the two files that deliver the entire privacy value proposition — stealth derivation/detection and the sponsored sweep. Neither has ever been exercised, because no UI calls either one and no test does either.

8. What is the strongest technical feature right now?

The data model and the ERC-6538/Announcer library code (`lib/registry.ts`, `lib/announcer.ts`, `lib/chain.ts`) are genuinely correct against the real standards — right function names, right ABIs, right canonical addresses. The overall architectural reasoning in the README (why AA/Paymaster is load-bearing, honest "what we don't provide" section) is also unusually clear-eyed for a hackathon README.

9. What claims can I safely make to judges?

"We designed a standards-correct ERC-5564/6538/4337 architecture and wrote the full library layer for it — registry, announcer, derivation, and sponsored-sweep logic." "We correctly identified

why Account Abstraction is necessary here, not decorative." "We know exactly what's broken and what's left, down to the line."

10. What claims should I NOT make?

Do not claim a working end-to-end flow, a working demo, working gas sponsorship, or "stealth address privacy" as something currently demonstrable — none of it currently executes. Do not claim "256× faster scanning" — that optimization isn't implemented. Do not describe demo mode as using "real" detection — its own code contradicts that.

11. Top 5 things to fix/build next.

(1) Add `lucide-react` so the app builds at all. (2) Fix the `StealthAddressType`/SDK mismatch in `lib/stealth.ts`. (3) Fix the four broken `permissionless` imports in `lib/smartAccount.ts`. (4) Build the setup/send/scan screens and wire the existing hooks to them. (5) Chain "derive → transfer → announce" into one actual send flow — right now those three steps exist as unconnected functions.

ONE-LINE SUMMARY

StealthTag has correctly designed what it wants to build and written most of the library-level plumbing for it, but as committed, zero screens work, the two files that would deliver the actual privacy guarantee crash on their first line of real use, and there is no automated test that would have caught either crash before a demo did.

Audit performed by reading every source file in the repository and cross-checking every claimed dependency call against the package actually present in `node_modules`, not against declared version ranges. No files were modified.